

µProcessador 6 Condicionais e Desvios

Inclua no circuito instruções de desvio condicional e não condicional, de acordo com o *assembly* do processador sorteado. Seu sorteio deve definir instruções *jump* (J, JP, JMP, JUMP) e instruções *branch* (B, BR, BRANCH) usando endereço de salto relativo¹ e absoluto², respectivamente, ou o contrário disso.

O processador deverá ser capaz de comparar dois números e descobrir qual deles é o menor. Pode alterar sua ULA se quiser.

Cuidado: o *debug* desta prática pode ser *brutal*, apesar da implementação ser simples. Mantenha a ordem no VHDL, no *assembly* e na simulação. Ajuda.

Implementação

Iremos implementar instruções de condicional típicas dos processadores CISC que, em geral, usam *flags*: elas são *flip-flops* independentes que guardam uma condição. Veja como funciona uma comparação no 8051:

```
CLR C      ; limpa flag C para não interferir na subtração
SUBB A,32  ; subtrai 32 do reg. A, setando aflag Z se iguais
JZ IGUAL   ; salta para IGUAL apenas se a flag Z está setada
DIF: NOP   ; entra nesta linha só se A ≠ 32
```

Para maior e menor é usada a *flag* C, de *carry*, e instrução JC (ou a de *overflow* ou a de sinal); as negativas (JNZ e JNC) também estão presentes. Revise o PDF sobre flags que está no Moodle.

Atenção: saltos relativos têm que pular tanto para frente como para trás (branch ou jump “-5” volta cinco instruções). *Utilize o operando em complemento de 2*. Se for feita extensão de sinal do operando, basta fazer normalmente a soma ao PC: ela vai funcionar, pelas propriedades de complemento de 2.

Os Flip-flops das Flags

Vamos reforçar: a ULA vai produzir os bits das flags (N, Z, C e/ou OV, de acordo com o seu sorteio) para qualquer conta que ela esteja fazendo. Mas *estes bits devem ser guardados em flip-flops*, externos à ULA, normalmente no top-level (talvez na unidade de controle).

Os flip-flops só vão ser atualizados na execução de instruções específicas da ULA (ADD, SUB, CMP, p.ex.) mas não devem ser atualizados em outras instruções (LD/MOV, JMP, branches, etc).

Para decidir se um salto condicional deve ser executado ou não, você deve consultar o valor de saída do flip-flop correspondente. Por exemplo, um BEQ só vai saltar se a saída do flip-flop Z estiver em 1, caso contrário não salta.

Consulte a tabela do lab#2 - ULA se você não lembra das condições sorteadas para a equipe.

Deteção de Estouro (Carry)

Esta seção é apenas caso vocês precisem do *carry flag* e ainda não tenham implementado.

Se estivermos lidando apenas com números não negativos, precisamos produzir apenas o *carry* (o vai-um da soma). Se trabalharmos com números sinalizados, podemos implementar o *overflow* (transbordamento de número sinalizado). Você pode incluir ambos se desejar.

Para detectar o *carry* numa soma em VHDL, somos obrigados a usar um bit adicional na

1 Usa delta de intruções a saltar: “branch 5” vai pular cinco instruções pra frente de onde ele está.

2 Usa o endereço destino como operando: “jump 34” vai pular para o endereço 34 na memória.

operação³. Veja um trecho da arquitetura:

```
signal in_a_17,in_b_17,soma_17: unsigned(16 downto 0);
begin
  in_a_17 <= '0' & in_a;      -- passamos in_a para 17 bits
  in_b_17 <= '0' & in_b;      -- idem in_b
  soma_17 <= in_a_17+in_b_17;
  carry_soma <= soma_17(16);  -- o carry eh o MSB da soma 17 bits
```

Se não usarmos números negativos, para determinar o *carry* na subtração “in_a – in_b” basta fazer uma comparação direta:

```
carry_subtr <= '0' when in_b<=in_a else -- só se ambos forem valores unsigned
               '1';
```

Para outras operações que você tenha implementado (multiplicação, incremento), os estouros podem ser ignorados, se você quiser.

Testes

Mantenha os pinos visíveis no *top level* como descritos no laboratório passado.

Para a entrega, o *testbench* e a ROM devem estar configurados para executar um programa que faz o seguinte:

- A. Carrega R3 (o registrador 3) com o valor 0
- B. Carrega R4 com 0
- C. Soma R3 com R4 e guarda em R4
- D. Soma 1 em R3
- E. Se R3<30 salta para a instrução do passo C
- F. Copia valor de R4 para R5

Sugiro fortemente escrever no papel (ou planilha) programas de teste, detalhando em cada linha a instrução em *assembly* e a codificação binária/hexadecimal e endereço na memória respectivos. Ajuda quando dá pau.

Se desejar, construa um montador simples que produz uma listagem hexadecimal a partir de um programa *assembly*.

Arquivos a Entregar

Anexe no Moodle:

- ✓ Arquivos .vhd compactados (inclua o arquivo .gtkw de sinais, caso você tenha usado);
- ✓ Especificação atualizada da codificação das instruções;
- ✓ Código *assembly* e codificação na ROM para o programa-teste pedido (pode estar apenas dentro do arquivo “rom.vhd” se você desejar, ou num arquivo à parte); repetindo: *eu quero as instruções em assembly do programa em ROM*.

Avaliação

- Se as instruções *assembly* estiverem não conformes com o sorteio:
 - -40% por erros graves presentes (não usou flags apesar de terem sido pedidas, não usou acumulador apesar de ter sido pedido);
 - -5% por cada erro leve presente (nome estranho em instrução, indicação ruim nos operandos, opcodes inconsistentes);
- Se não funcionar o programa, mas o VHDL parece ok: -20%

³ Lembrete: a concatenação é o operador &.

- Se executar instrução seguinte ao desvio: -10%
- Se a constante do salto não estiver em complemento de dois: -10%
- Se o salto não for absoluto/relativo como especificado: -10%

Caso o sorteio da sua equipe contradiga algo da avaliação, fique livre para escolher este particular.

O professor irá verificar o funcionamento pessoalmente em sala na aula seguinte à entrega deste lab.

Observações sobre o sorteio [Replay]

Observe que, no caso de desvios relativos, a constante do “delta” deverá estar **obrigatoriamente em complemento de 2** – não pode estar em sinal-magnitude. (Pode continuar usando signals UNSIGNED normalmente no VHDL, vai funcionar, confie).